User Profile Service Architecture

Document date: 2023-06-12 Classification: Internal

Table of Contents

1. Overview 2
2. Data Stores 3
3. Dependencies 4
4. Security and Authentication 5
5. Scaling 6
6. Open Questions 7

1. Overview

The User Profile Service (user-profile-service) is owned by the Customer Data team, with Hannah Weber as technical lead. It is implemented in Kotlin on Ktor and listens on port 8081. Feature work is paused during incident follow-up weeks until action items are closed. All services must expose Prometheus metrics on the /metrics path. Load tests should be coordinated in the platform channel so they do not trip alerts.

Every service has a README that links to its dashboards and runbook. Service-level objectives are reviewed quarterly with product owners. Database schema migrations must be backward compatible for at least one release. Feature work is paused during incident follow-up weeks until action items are closed.

2. Data Stores

Primary data store: see data store notes. Staging mirrors production topology but runs at roughly one tenth of the capacity. The dashboards were updated to show saturation alongside latency and errors.

User Profile Service stores profiles in self-managed MySQL 5.7 (profile-db). Read replicas serve profile lookups.

3. Dependencies

User Profile Service depends on: auth-service, profile-db, S3 bucket acme-avatars. Feature work is paused during incident follow-up weeks until action items are closed. The dashboards were updated to show saturation alongside latency and errors.

4. Security and Authentication

Authentication mechanism: User Profile Service uses JWT bearer tokens validated at the api-gateway. Staging mirrors production topology but runs at roughly one tenth of the capacity. Every service has a README that links to its dashboards and runbook.

5. Scaling

Service-level objectives are reviewed quarterly with product owners. Remember to tag pull requests with the owning team so reviews get routed correctly. Structured JSON logging is mandatory; free-text logs are dropped by the log pipeline. Postmortems are blameless and are published within five business days. Staging mirrors production topology but runs at roughly one tenth of the capacity.

Structured JSON logging is mandatory; free-text logs are dropped by the log pipeline. Dependency upgrades are batched weekly by the platform bot. Every service has a README that links to its dashboards and runbook. Load tests should be coordinated in the platform channel so they do not trip alerts.

6. Open Questions

Capacity planning for the holiday peak starts in September every year. Service-level objectives are reviewed quarterly with product owners. Capacity planning for the holiday peak starts in September every year.